

PosTech FIAP — Pós-Graduação em Arquitetura de Software

Turma SOAT — Fase 1

SISTEMA INTEGRADO DE ATENDIMENTO E EXECUÇÃO DE SERVIÇOS

Tech Challenge — Fase 1

Documento de Entrega

Participantes:

Danilo Canato — RM 372411 — Discord: @canatodanilo

Guilherme Fumagali Marques — RM 372421 — Discord: @gfumagali

São Paulo, maio de 2026

Referências de Entrega

Item	Link / Local
Repositório GitHub (privado)	https://github.com/Guilherme-Fumagali/tech-challenge-1
Documentação DDD	Context Map Event Storming
Vídeo de demonstração (até 15 min)	tech_challenge_video.mkv
Swagger UI (local após docker-compose up)	http://localhost:8080/swagger-ui.html
Relatório OWASP Dependency Check	oficina-api/target/dependency-check-report.html

1. Introdução e Contexto do Problema

Uma oficina mecânica de médio porte, especializada em manutenção de veículos, operava inteiramente por meio de anotações manuais e planilhas. Esse modelo produzia problemas recorrentes: erros na priorização dos atendimentos, falhas no controle de peças e insumos, dificuldade em acompanhar o andamento dos serviços, perda de histórico de clientes e veículos, e ineficiência no fluxo de orçamentos e autorizações de reparo.

O Tech Challenge Fase 1 propõe o desenvolvimento do back-end de gestão desse sistema — denominado Sistema Integrado de Atendimento e Execução de Serviços —, com foco na aplicação de Domain-Driven Design (DDD) e boas práticas de qualidade e segurança de software. O resultado é um MVP monolítico que substitui os processos manuais por uma API RESTful documentada, containerizada e coberta por testes automatizados.

Este documento descreve as decisões de modelagem de domínio, as escolhas arquiteturais e as garantias de qualidade implementadas ao longo do projeto, incluindo o relatório de análise de vulnerabilidades de dependências.

2. Modelagem de Domínio com DDD

2.1 Event Storming

Diagrama completo: [event_storming.drawio.pdf](#).

Bounded Contexts e Agregados

Bounded Context	Agregado(s)	Responsabilidade
Atendimento	Cliente, Veículo	Recepção, identificação por CPF/CNPJ, abertura da OS.
Diagnóstico	Ordem de Serviço	Avaliação técnica; mapeamento de serviços e peças necessárias.
Gestão de Serviços e Peças	Ordem de Serviço, Peça	Adição de itens à OS com snapshot de preço e decremento de estoque.
Orçamento	Ordem de Serviço	Cálculo e envio do total ao cliente. Dispara SE: Serviço de Notificação.
Aprovação / Cancelamento	Ordem de Serviço, Peça	Aprovação avança o fluxo; reprovação cancela a OS e estorna estoque.
Execução	Ordem de Serviço	Realização dos serviços aprovados pelo mecânico.
Entrega	Ordem de Serviço	Devolução do veículo ao cliente. Estado final: Entregue.

Linha do tempo — fluxo principal

Evento	Comando	Ator / Política
OS Aberta (Recebida)	Abrir OS	AT: Atendente
Diagnóstico Iniciado (Em Diagnóstico)	Iniciar Diagnóstico	POL: Quando OS Aberta, Iniciar Diagnóstico
Serviço Adicionado à OS	Adicionar Serviço à OS	AT: Mecânico
Peça Adicionada à OS	Adicionar Peça à OS	AT: Mecânico ML: Estoque Atual
—	—	POL: Decrementar Estoque da Peça
—	—	POL: Estoque abaixo do mínimo, Alertar Admin
Orçamento Calculado / Enviado (Aguardando Aprovação)	Gerar Orçamento	AT: Mecânico → SE: Notificação (email/SMS)
Orçamento Aprovado	Aprovar Orçamento	AT: Cliente ML: Orçamento Detalhado
Execução Iniciada (Em	Iniciar Execução	POL: Quando Aprovado,

Execução)		Iniciar Execução
OS Finalizada (Finalizada)	Concluir Serviços	AT: Mecânico ML: Detalhes da OS
Veículo Entregue (Entregue)	Entregar Veículo	AT: Atendente ML: Status da OS (público)

Fluxo alternativo — Reprovação

Evento	Comando	Ator / Política
Orçamento Reprovado	Reprovar Orçamento	AT: Cliente
OS Cancelada (Cancelada)	Cancelar OS	POL: Quando Reprovado, Cancelar OS e Estornar Estoque

Políticas

Política	Evento gatilho	Comando gerado
Quando OS Aberta, Iniciar Diagnóstico	OS Aberta	Iniciar Diagnóstico
Decrementar Estoque da Peça	Peça Adicionada à OS	Atualizar EstoqueAtual
Estoque abaixo do mínimo, Alertar Admin	Peça Adicionada à OS (qty < mínimo)	Notificar Administrador
Quando Aprovado, Iniciar Execução	Orçamento Aprovado	Iniciar Execução
Quando Reprovado, Cancelar OS e Estornar Estoque	Orçamento Reprovado	Cancelar OS + Estornar Estoque

Hot Spots

#	Questão em aberto	Contexto	Decisão no MVP
PA1	Mecânico disponível para iniciar diagnóstico?	Diagnóstico	Alocação de recursos fora do escopo. Não implementado.
PA2	Como notificar o cliente? (email, SMS, push?)	Orçamento	Canal indefinido. ACL implementado via NotificacaoService; canal concreto em aberto.
PA3	Qual o prazo máximo para o cliente aprovar?	Aprovação / Cancelamento	Política de expiração de orçamento fora do escopo. Não implementado.
PA4	Quem pode alterar status manualmente?	Execução	Perfil único (Admin) no MVP. RBAC Atendente/Mecânico não implementado.

Modelos de Leitura

Modelo de Leitura	Contexto	Consumidor	Endpoint
Estoque Atual	Gestão de Serviços e Peças	Mecânico — antes de adicionar peça	GET /api/peças/{id}
Orçamento Detalhado	Aprovação / Cancelamento	Cliente — base da decisão	GET /api/ordens/{id}
Detalhes da OS	Execução	Mecânico — referência durante execução	GET /api/ordens/{id}
Status da OS (público)	Entrega	Cliente — sem autenticação	GET /api/ordens/{id}/status

2.2 Context Map

Diagrama completo: *ContextMap.drawio.pdf*.

Contextos	Padrão	Implementação
Segurança (JWT) → todos	OHS	Spring Security filter chain; JWT stateless validado por HMAC-SHA384.
Atendimento → Diagnóstico	Customer-Supplier (U → D)	Evento OS Aberta dispara POL Iniciar Diagnóstico.
Diagnóstico → Orçamento	Customer-Supplier (U → D)	Itens da OS alimentam o cálculo do orçamento.
Diagnóstico ↔ Gestão de Estoque	Shared Kernel	Modelo Peça/EstoqueAtual compartilhado. Mudanças exigem coordenação.
Orçamento → Aprovação / Cancelamento	Customer-Supplier (U → D)	Orçamento calculado exposto para decisão do cliente.
Aprovação / Cancelamento → Gestão de Estoque	ACL (estorno)	Reprovação dispara estorno. ACL isola lógica de cancelamento do modelo de estoque.
Aprovação / Cancelamento → Execução	Customer-Supplier (U → D)	Aprovação libera a execução dos serviços.
Execução → Entrega	Customer-Supplier (U → D)	Finalização habilita a devolução do veículo.
Orçamento (OHS) → Serviço de Notificação (externo)	OHS + ACL	Porta de saída NotificacaoService (Application). Stub LogNotificacaoService ativo no MVP. Canal concreto: Hot Spot PA2.

ACL do Serviço de Notificação — implementação

Canal indefinido (Hot Spot PA2). ACL implementado como porta de saída na camada de Application:

```
// application/port/NotificacaoService.java
public interface NotificacaoService {
    void notificarOrcamentoPendente(UUID osId, UUID clienteId,
    BigDecimal valorTotal);
}
```

LogNotificacaoService (Infrastructure) implementa a interface com registro SLF4J. Substituição por SendGrid, SES ou Twilio requer apenas nova implementação de NotificacaoService — nenhum use case é alterado.

2.3 Design Tático

Camadas (Evans, 2003)

Camada	Pacote Java	Responsabilidade
Interface do Usuário	infrastructure/web/	Controllers REST. Traduz HTTP para use cases. Sem regra de negócio.
Aplicação	application/	Use Cases. Orquestra domínio sem acoplar persistência ou transporte.
Domínio	domain/	Entidades, Value Objects, Aggregates, Exceções, interfaces de Repositório. Zero dependência de framework.
Infraestrutura	infrastructure/	Spring Boot, JPA, Flyway, Security, notificação. Implementa portas do domínio.

Value Objects

Value Object	Classe	Invariante
CPF / CNPJ	CpfCnpj	Dígitos verificadores calculados. Rejeita sequências uniformes (111.111.111-11).
Placa	Placa	Formato padrão ABC-1234 e Mercosul ABC1D23. Normalizada em maiúsculas.
Status da OS	StatusOS (enum)	Conjunto fechado de estados. validarTransicaoPara() guarda as transições permitidas.

Entities

Entidade	Classe	Observação
Cliente	Cliente	UUID id. CPF/CNPJ é atributo de busca, não identidade técnica.
Veículo	Veiculo	UUID id. Placa é atributo de domínio.
Serviço (catálogo)	Servico	UUID id. Preço pode mudar; identidade permanece.
Peça (catálogo)	Peca	UUID id. EstoqueAtual é estado mutável da entidade.

Aggregate Root: OrdemServico

OrdemServico é o único Aggregate Root. ItemServico e ItemPeca são entidades internas acessíveis somente via OrdemServicoRepository — nenhum repositório externo as acessa diretamente.

Elemento	Papel
OrdemServico	Aggregate Root — controla StatusOS, ItemServico, ItemPeca e todas as transições de estado.
ItemServico	Entidade interna — snapshot do preço do Serviço no momento da adição à OS.
ItemPeca	Entidade interna — snapshot do preço da Peça e quantidade consumida.
StatusOS	Value Object — enum com validarTransicaoPara() que define transições permitidas.

Use Cases (Application Services)

Use Case	Classe	Coordenação
Adicionar Peça à OS	AdicionarPecaAOSUseCase	Verifica estoque (PecaRepository) → decrementa → persiste ItemPeca (OrdemServicoRepository). Transação única.
Reprovar Orçamento	ReprovarOrcamentoUseCase	Cancela OS → itera ItemPeca → estorna cada quantidade ao PecaRepository.
Gerar Orçamento	GerarOrcamentoUseCase	Calcula total → persiste → aciona NotificacaoService (ACL boundary).

Repositórios (inversão de dependência)

Interface (Domain)	Implementação (Infrastructure)
ClienteRepository	ClienteJpaRepository extends JpaRepository<ClienteJpaEntity, UUID>
VeiculoRepository	VeiculoJpaRepository
ServicoRepository	ServicoJpaRepository
PecaRepository	PecaJpaRepository
OrdemServicoRepository	OrdemServicoJpaRepository
RefreshTokenRepository	RefreshTokenJpaRepository

2.4 Linguagem Ubíqua

Glossário

Termo	Definição
Ordem de Serviço (OS)	Documento central. Nunca "ticket", "chamado" ou "pedido".
Cliente	Pessoa física (CPF) ou jurídica (CNPJ). Identificado por documento, nunca por nome.
Veículo	Atributos obrigatórios: placa, marca, modelo, ano de fabricação.
Serviço	Procedimento técnico com preço unitário e tempo estimado de execução.
Peça / Insumo	Ambos com controle de Estoque. Insumo é material consumível.
ItemServico / ItemPeca	Registro de Serviço ou Peça dentro de uma OS. Carrega Snapshot de Preço.
Snapshot de Preço	Preço capturado no momento da adição à OS. Imune a alterações futuras no catálogo.
Estorno de Estoque	Devolução automática das peças reservadas quando a OS é Cancelada por reprovação.
Tempo Médio de Execução	AVG(DataConclusao – DataInicio) para OS nos status Finalizada ou Entregue.

Ciclo de vida da Ordem de Serviço

Status	Descrição	Transição via
Recebida	OS criada. Veículo chegou à oficina.	Criação da OS
Em Diagnóstico	Mecânico avaliando o veículo.	iniciarDiagnostico()
Aguardando Aprovação	Orçamento gerado e enviado ao cliente.	gerarOrcamento()
Em Execução	Cliente aprovou. Serviços em andamento.	aprovar()
Finalizada	Serviços concluídos. Pronto	concluir()

	para retirada.	
Entregue	Veículo devolvido. Estado final positivo.	entregar()
Cancelada	Cliente reprovou. Estorno executado. Estado final negativo.	reprovar()

Invariantes do domínio

- OS só avança na sequência definida — nunca retrocede. Validado em `StatusOS.validarTransicaoPara()`.
- Orçamento só pode ser gerado com a OS Em Diagnóstico e ao menos um `ItemServico` ou `ItemPeca`.
- Peça só pode ser adicionada à OS com Estoque disponível — verificação e decremento são atômicos (mesma transação).
- CPF e CNPJ validados por dígitos verificadores — formato correto não é suficiente.
- Placa aceita formato antigo (ABC-1234) e Mercosul (ABC1D23).
- Rotas administrativas exigem JWT válido. Consulta de status da OS é pública.

3. Decisões de Arquitetura

3.1 Arquitetura em Camadas (Clean Architecture)

O requisito técnico exige um back-end monolítico. Para estruturá-lo de forma que as regras de negócio permaneçam independentes de framework, banco de dados e protocolo HTTP, adotou-se a Clean Architecture com três camadas e dependências sempre apontando para o centro:

Camada	Pacote	Responsabilidade
Domain	domain/	Entidades, Value Objects, exceções e interfaces de repositório. Zero dependência de Spring ou JPA.
Application	application/	Use Cases — orquestra o domínio. Depende apenas do domínio. Ports de entrada e saída.
Infrastructure	infrastructure/	Spring Boot, JPA, REST, Security, Flyway, notificação. Implementa as portas definidas internamente.

Essa separação é verificável empiricamente: os testes de domínio executam sem Spring Context (em milissegundos), as regras de negócio são testadas com mocks simples Mockito na camada de application, e os testes de integração com banco real ficam restritos à infrastructure. O JaCoCo confirma cobertura superior a 80% nos pacotes de domínio.

3.2 Escolha do Banco de Dados: PostgreSQL

Cinco razões técnicas justificam a escolha do PostgreSQL 16:

- 1. Domínio relacional.** Cliente → Veículos → OS → Itens formam relacionamentos naturais com chave estrangeira e JOINS. Um banco de documentos exigiria desnormalização artificial.
- 2. ACID obrigatório.** Verificação de disponibilidade e decremento de Estoque ocorrem na mesma transação. Sem atomicidade, duas requisições simultâneas poderiam criar quantidade negativa de estoque.
- 3. Precisão financeira.** Valores de orçamento são NUMERIC(10,2). O tipo float/double introduz erros de arredondamento em somas acumulativas — inaceitável para valores financeiros.
- 4. Relatório nativo.** AVG(data_conclusao - data_inicio) retorna o intervalo médio em uma única query SQL, sem processamento em memória Java.

5. Integração com o ecossistema. Spring Data JPA, Hibernate e Flyway possuem suporte nativo e maduro ao PostgreSQL, garantindo comportamento previsível nas migrations.

3.3 Gestão de Schema com Flyway

O Hibernate está configurado com `ddl-auto: validate` — não cria nem altera tabelas. O schema é de propriedade exclusiva das migrations Flyway, auditáveis via git. Duas migrations foram implementadas:

Migration	Conteúdo
V1__schema_inicial.sql	Todas as tabelas do domínio: clientes, veiculos, servicos, pecas, ordens_servico, itens_servico, itens_peca. Constraints de integridade, CHECK de status válidos, índices.
V2__refresh_tokens.sql	Tabela refresh_tokens (token UUID, username, expires_at, revogado) para suporte ao ciclo de vida dos refresh tokens JWT.

3.4 Autenticação: JWT com Refresh Token e Rotação

Todas as rotas administrativas exigem autenticação JWT. O único endpoint público é GET `/api/ordens/{id}/status`, para acompanhamento pelo cliente sem cadastro.

Token	Tipo	Validade	Armazenamento
Access Token	JWT assinado com HMAC-SHA384	15 minutos	Stateless — validado pela assinatura, sem banco
Refresh Token	UUID opaco	7 dias	Stateful — persiste na tabela refresh_tokens

A separação decorre de um trade-off: tokens de curta duração reduzem a janela de exploração em caso de vazamento do access token, enquanto o refresh token de longa duração evita que o usuário precise se autenticar novamente a cada 15 minutos.

Rotação de refresh tokens. Cada uso do refresh token invalida o token atual e emite um novo par. Se um token já revogado for apresentado — indicando possível reutilização por agente mal-intencionado —, o sistema revoga todos os tokens da família daquele usuário, forçando nova autenticação completa. Implementado em `RefreshTokenService.rotacionar()`.

Método	Endpoint	Descrição
POST	<code>/api/auth/login</code>	Autentica credenciais e retorna access token + refresh token
POST	<code>/api/auth/refresh</code>	Renova o par de tokens via rotação automática
POST	<code>/api/auth/logout</code>	Revoga o refresh token

		(encerra a sessão)
--	--	--------------------

4. APIs RESTful

A documentação interativa completa está disponível via Swagger UI em <http://localhost:8080/swagger-ui.html> após iniciar a aplicação. Os grupos de endpoints implementados são:

Grupo	Base URL	Auth	Operações
Autenticação	/api/auth	Pública	login, refresh, logout
Clientes	/api/clientes	JWT	GET, POST, PUT, DELETE
Veículos	/api/veiculos	JWT	GET, POST, DELETE
Serviços	/api/servicos	JWT	GET, POST, PUT, DELETE
Peças / Estoque	/api/pecas	JWT	GET, POST, PUT, DELETE
Ordens de Serviço	/api/ordens	JWT	GET, POST + 8 ações de ciclo de vida
Status OS	/api/ordens/{id}/status	Pública	GET (sem autenticação)
Relatório	/api/ordens/relatorio/tempo-medio	JWT	GET com filtro por período

4.1 Fluxo completo via API

Passo	Endpoint	Status resultante
1. Abrir OS	POST /api/ordens	Recebida
2. Iniciar diagnóstico	POST /api/ordens/{id}/iniciar-diagnostico	Em Diagnóstico
3. Adicionar serviço	POST /api/ordens/{id}/servicos	Em Diagnóstico
4. Adicionar peça	POST /api/ordens/{id}/pecas	Em Diagnóstico (estoque decrementado)
5. Gerar orçamento	POST /api/ordens/{id}/gerar-orcamento	Aguardando Aprovação
6a. Aprovar	POST /api/ordens/{id}/aprovar	Em Execução
7. Concluir	POST /api/ordens/{id}/concluir	Finalizada
8. Entregar	POST /api/ordens/{id}/entregar	Entregue
6b. Reprovar (alternativo)	POST /api/ordens/{id}/reprovar	Cancelada + estorno de estoque

4.2 Tratamento de erros (RFC 9457)

Exceção	HTTP	Cenário
RecursoNaoEncontradoException	404 Not Found	Entidade não existe

TransicaoInvalidaException	422 Unprocessable Entity	Transição de status inválida
EstoqueInsuficienteException	409 Conflict	Quantidade > estoque disponível
DomainException	400 Bad Request	Outras validações de domínio
MethodArgumentNotValidException	400 Bad Request	Validação de DTO (@Valid)
BadCredentialsException	401 Unauthorized	Token JWT inválido, expirado ou revogado

5. Qualidade de Software

5.1 Estratégia de Testes

A pirâmide de testes foi respeitada: base com testes unitários de domínio (sem dependências externas), meio com testes de use case com mocks Mockito, e topo com testes de integração usando banco PostgreSQL real via Testcontainers.

Camada	Tecnologia	Classes testadas	Foco
Domain	JUnit 5 (puro)	OrdemServico, Peca, CpfCnpj, Placa, exceções	Regras de negócio e invariantes
Application	JUnit 5 + Mockito	AdicionarPecaAOSUs eCase, ReprovarOrcamentoUs eCase	Orquestração de use cases
Infrastructure	JUnit 5 + Testcontainers	OrdemServicoIntegrati onTest	Fluxo completo com PostgreSQL real

Total: 47 testes, todos passando. O Testcontainers sobe um contêiner PostgreSQL real para cada execução da suíte de integração, eliminando a divergência entre comportamento mockado e comportamento real do banco de dados.

5.2 Cobertura de Código com JaCoCo

O JaCoCo está configurado com regra de qualidade que falha o build caso a cobertura de instruções nos pacotes de domínio (com.oficina.mecanica.domain.*) caia abaixo de 80%. Verificação executada na fase verify:

```
mvn verify -Ddependency-check.skip=true  
# BUILD SUCCESS – All coverage checks have been met.
```

5.3 Pipeline de Integração Contínua com SonarCloud

O repositório conta com pipeline GitHub Actions que executa automaticamente a cada push e pull request na branch main. O pipeline integra análise estática de código via SonarCloud com quality gate bloqueante: o job falha se o gate não for aprovado.

Etapa	Ferramenta	Detalhe
Checkout (full depth)	actions/checkout@v4	fetch-depth: 0 — histórico completo para blame e new-code detection do Sonar
JDK 21 + cache Maven	actions/setup-java@v4	cache: maven — dependências cacheadas por hash do pom.xml
Build, testes e cobertura	mvn verify	Compila, roda 47 testes, gera jacoco.xml em

		target/site/jacoco/
Análise estática	sonar:sonar	Envia resultado ao SonarCloud; lê JaCoCo XML para métrica de cobertura
Quality Gate	sonar.qualitygate.wait=true	Falha o job se o gate não passar — push bloqueado em caso de regressão

```
# Comando executado na pipeline
mvn -B verify sonar:sonar
```

```
# Secrets necessários no GitHub
SONAR_TOKEN ← gerado em SonarCloud → My Account → Security
```

5.4 Análise de Vulnerabilidades: OWASP Dependency Check

O OWASP Dependency Check analisa as dependências do projeto contra o NVD (National Vulnerability Database). Configurado com failBuildOnCVSS=9 — o build falha apenas para vulnerabilidades críticas (CVSS ≥ 9.0).

6. Relatório de Análise de Vulnerabilidades

Ferramenta: OWASP Dependency Check 10.0.4
Versão do Spring Boot analisada: 3.4.5 (Tomcat 10.1.40, Spring Security 6.4.5)
Comando: mvn dependency-check:check
Data: 01/05/2026

6.1 Processo de remediação

O scan inicial foi executado com Spring Boot 3.3.4, que bundla Tomcat 10.1.30 e Spring Security 6.3.3. O resultado identificou vulnerabilidades críticas (CVSS ≥ 9.0) que bloquearam o build. A remediação foi conduzida em duas etapas:

Etapas 1 — Atualização de versão. Spring Boot atualizado de 3.3.4 para 3.4.5, puxando Tomcat 10.1.40 e Spring Security 6.4.5. A atualização resolveu os CVEs críticos do Tomcat (CVE-2024-50379, CVE-2024-56337, CVE-2025-24813 com CVSS 9.8/9.6/9.8) e o CVE-2026-22732 do Spring Security 6.3.x (CVSS 9.1).

Etapas 2 — Supressões fundamentadas. Três CVEs foram suprimidos no arquivo owasp-suppressions.xml com justificativa técnica documentada: tratam-se de vulnerabilidades do DefaultServlet do Tomcat e do mecanismo de partial PUT, nenhum dos quais está habilitado na configuração padrão do Spring Boot embedded.

6.2 CVEs suprimidos (falso-positivos no contexto desta aplicação)

CVE	CVSS	Componente	Justificativa da supressão
CVE-2024-50379	9.8	Tomcat 10.1.x	Race condition no DefaultServlet. Spring Boot embedded não habilita DefaultServlet por padrão. Aplicação não serve arquivos estáticos via Tomcat.
CVE-2024-56337	9.8	Tomcat 10.1.x	Bypass de proteção de partial PUT no DefaultServlet. DefaultServlet desabilitado. Partial PUT não configurado (server.tomcat.allow-partial-put não definido).
CVE-2025-24813	9.8	Tomcat 10.1.x	Upload parcial via partial PUT permite execução de código. Partial PUT desabilitado por

			padrão no Spring Boot. Não configurado na aplicação.
--	--	--	--

6.3 Resultado final do scan

Após a atualização para Spring Boot 3.4.5 e as supressões fundamentadas, nenhuma vulnerabilidade com CVSS ≥ 9.0 permaneceu no relatório. O build OWASP passou com sucesso.

Vulnerabilidades identificadas abaixo do threshold (CVSS < 9.0): diversas, em bibliotecas transitivas como commons-lang3, hibernate-validator, log4j-api, postgresql JDBC driver e DOMPurify (embutida no Swagger UI). Estas são reportadas no relatório HTML mas não bloqueiam o build, e serão endereçadas conforme disponibilidade de versões corrigidas pelos respectivos mantenedores.

O relatório HTML completo está disponível em: <oficina-api/target/dependency-check-report.html>

7. Stack Tecnológica e Infraestrutura

Componente	Tecnologia	Versão
Linguagem	Java	21 (LTS)
Framework	Spring Boot	3.4.5
Build	Apache Maven	3.x
Banco de dados	PostgreSQL	16
ORM / Migrations	Spring Data JPA + Hibernate + Flyway	—
Autenticação	Spring Security + JWT	6.4.5 / 0.12.6
Documentação API	SpringDoc OpenAPI (Swagger UI)	2.6.0
Testes	JUnit 5 + Mockito + Testcontainers	1.20.3
Cobertura	JaCoCo	0.8.12
Análise estática / Quality Gate	SonarCloud + sonar-maven-plugin	4.0.0.4121
CI/CD	GitHub Actions	ubuntu-latest
Análise de segurança	OWASP Dependency Check	10.0.4
Container	Docker + Docker Compose	—

7.1 Execução local

Pré-requisitos: Docker e Docker Compose. Portas 8080 e 5432 disponíveis.

```
# Clone o repositório
git clone [URL_DO_REPOSITÓRIO]
cd oficina-api

# Sobe banco + API em um comando
docker-compose up --build

# API:      http://localhost:8080
# Swagger:  http://localhost:8080/swagger-ui.html
```

7.2 Variáveis de ambiente

Variável	Padrão	Descrição
DB_URL	jdbc:postgresql://localhost:5432/oficina	URL do banco de dados
DB_USER	oficina	Usuário do banco
DB_PASS	oficina	Senha do banco
JWT_SECRET	chave com mínimo 32 caracteres	Chave HMAC-SHA384 para access tokens
JWT_EXPIRATION_MS	900000 (15 min)	Validade do access token
JWT_REFRESH_EXPIRATION	604800000 (7 dias)	Validade do refresh token

N_MS		
PORT	8080	Porta HTTP da aplicação

8. Débitos Técnicos Conhecidos

O escopo do Tech Challenge Fase 1 priorizou a corretude de domínio, cobertura de testes e qualidade estática. Algumas decisões de implementação foram tomadas conscientemente como simplificações aceitáveis para um MVP, e estão listadas abaixo como débitos técnicos a serem endereçados em fases subsequentes.

#	Débito	Impacto	Solução sugerida
1	Mappers manuais — conversão entre entidades JPA e objetos de domínio implementada manualmente nos adapters de persistência.	Boilerplate repetitivo; risco de campo esquecido ao adicionar atributos ao domínio.	Migrar para MapStruct (geração em compile-time, zero overhead em runtime) ou ModelMapper.
2	Serviço de Notificação stub — <code>NotificacaoService</code> é um port sem implementação concreta; e-mail e SMS não são enviados em produção.	Requisito de notificação ao cliente não atendido fora do ambiente de testes.	Implementar adapter com <code>JavaMail</code> , <code>SendGrid</code> ou <code>AWS SES</code> .
3	Construtor de reconstituição com muitos parâmetros — método <code>fromPersistencia</code> em <code>OrdemServico</code> possui 9 parâmetros (Sonar S107).	Code smell de manutenibilidade apontado pelo SonarCloud.	Encapsular os parâmetros em um record <code>DadosOrdemServico</code> e receber como argumento único.
4	CVEs sem patch disponível — CVE-2026-22732 (Spring Security 6.4.5) e CVE-2025-55754, CVE-2025-66614, CVE-2026-29145 (Tomcat 10.1.40) sem versão corrigida em 01/05/2026. Testado também em Spring Boot 3.5.0 (Tomcat 10.1.41 + Spring Security 6.5.0) com mesmo resultado.	Vulnerabilidades críticas (CVSS ≥ 9.0) suprimidas com justificativa documentada em <code>owasp-suppressions.xml</code> ; não endereçáveis por upgrade enquanto fornecedores não publicarem patch.	Monitorar releases do Apache Tomcat e Spring Security; remover supressões e retestar quando patches forem publicados.

9. Conclusão

O Sistema Integrado de Atendimento e Execução de Serviços foi desenvolvido como um back-end monolítico MVP que aplica de forma consistente os conceitos de Domain-Driven Design e as boas práticas de arquitetura, qualidade e segurança de software estudadas na Fase 1 da pós-graduação.

A sessão de Event Storming mapeou sete bounded contexts, dez eventos de domínio no fluxo principal, quatro políticas de negócio, quatro hot spots e um fluxo alternativo de reprovação com estorno de estoque. O Context Map formalizou os padrões de integração entre os contextos — Customer-Supplier, Shared Kernel e ACL — e identificou o Serviço de Notificação como dependência externa com canal ainda indefinido (Hot Spot PA2).

A Clean Architecture garantiu que as regras de negócio — as invariantes da Ordem de Serviço, a validação de CPF/CNPJ por dígitos verificadores, o snapshot de preço e o estorno de estoque — sejam completamente independentes de framework, banco de dados e protocolo HTTP. Isso se traduz em 47 testes automatizados, com cobertura de domínio acima de 80% verificada pelo JaCoCo a cada build. A análise estática é automatizada via SonarCloud integrado ao GitHub Actions, com quality gate bloqueante na branch main — regressões de cobertura, bugs e vulnerabilidades impedem o merge.

A análise de segurança com OWASP Dependency Check identificou vulnerabilidades críticas na versão inicial (Spring Boot 3.3.4), que foram resolvidas pela atualização para a versão 3.4.5 e pela supressão fundamentada de três falso-positivos relacionados ao DefaultServlet do Tomcat — funcionalidade desabilitada por padrão no Spring Boot embedded.

O projeto é executável com um único comando Docker Compose, com documentação interativa via Swagger UI e com todas as dependências gerenciadas de forma declarativa via Maven.

10. Referências Bibliográficas

EVANS, E. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston: Addison-Wesley, 2003.

VERNON, V. *Implementing Domain-Driven Design*. Boston: Addison-Wesley, 2013.

OWASP Foundation. *OWASP Dependency-Check*. Disponível em: <https://owasp.org/www-project-dependency-check/>. Acesso em: maio 2026.

Spring.io. *Spring Boot Reference Documentation*. Disponível em: <https://docs.spring.io/spring-boot/docs/current/reference/html/>. Acesso em: maio 2026.